

Practical Assignment 03

Performance Evaluation of Regression Models

Application: California House Price Prediction

Subject: Artificial Intelligence and Machine Learning (AIML)

Name	Sakshi Kailas Shewale
PRN	202401040126
Roll No.	31
Division	A
Batch	A2

1. Aim and Objectives

To develop and evaluate regression models for a real-world application (house price prediction) and to implement and analyse Gradient Descent optimisation for Linear Regression.

- Build four regression models (Linear, Ridge, Lasso, Random Forest) on the California Housing dataset.
- Evaluate them using MAE, MSE, RMSE, R^2 , Adjusted R^2 and 5-fold cross-validation.
- Implement Gradient Descent for Linear Regression from scratch, study the effect of the learning rate, and compare it with the closed-form Normal Equation and scikit-learn.

2. Problem Statement and Dataset

Accurate house price estimation supports buyers, sellers, banks and urban planners. This is a supervised regression problem: predict the median house value of a census block group from its demographic and geographic features.

Dataset: California Housing (scikit-learn, derived from the 1990 US Census). It has **20,640 samples**, **8 input features** and one target, **MedHouseVal** (in units of \$100,000). There are no missing values.

Feature	Description
MedInc	Median income in the block group (tens of thousands of dollars)
HouseAge	Median age of houses in the block group
AveRooms / AveBedrms	Average number of rooms / bedrooms per household
Population	Block group population
AveOccup	Average number of household members
Latitude / Longitude	Geographic location of the block group

2.1 Exploratory Data Analysis

The summary statistics show heavy outliers in some features: AveRooms has a maximum of about 142, AveOccup about 1243 and Population about 35,682, against typical values of 5, 3 and 1,400. The target is capped at 5.0 (\$500,000), so the most expensive blocks are censored. The correlation heatmap shows that MedInc has the strongest positive correlation with house value, while Latitude and Longitude are strongly (negatively) correlated with each other, which indicates possible multicollinearity.

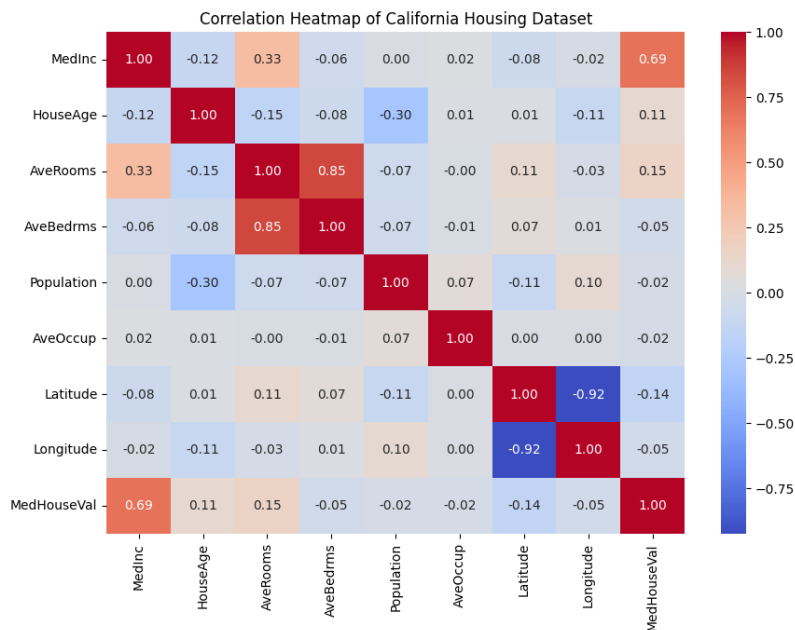


Figure 1: Correlation heatmap of the California Housing dataset

3. Methodology

- **Split:** 80% training (16,512 samples) and 20% testing (4,128 samples), random_state = 42.
- **Scaling:** StandardScaler fitted on the training set only (to avoid data leakage) and applied to the test set. Scaling is essential for Ridge/Lasso regularisation and for Gradient Descent convergence. Random Forest uses unscaled data, as trees are scale-invariant.
- **Linear Regression:** baseline model; assumes a linear relationship between features and price.
- **Ridge (L2, alpha = 1.0):** shrinks coefficients; handles multicollinearity.
- **Lasso (L1, alpha = 0.001):** can shrink coefficients to exactly zero, which gives built-in feature selection.
- **Random Forest (100 trees):** ensemble of decision trees that captures non-linear relationships and feature interactions.

3.1 Evaluation Metrics

Metric	Formula	Interpretation
MAE	$(1/n) \sum y - \hat{y} $	Average absolute error (same unit as target)
MSE	$(1/n) \sum (y - \hat{y})^2$	Penalises large errors heavily
RMSE	$\sqrt{\text{MSE}}$	Error in target units; sensitive to outliers

Metric	Formula	Interpretation
R^2	$1 - SS_{res} / SS_{tot}$	Proportion of variance explained
Adjusted R^2	$1 - (1 - R^2)(n - 1)/(n - p - 1)$	R^2 penalised for number of features p

4. Task 1: Regression Model Results

Model	MAE	MSE	RMSE	R^2	Adj. R^2	CV R^2
Linear Regression	0.5332	0.5559	0.7456	0.5758	0.5750	0.5530
Ridge Regression	0.5332	0.5559	0.7456	0.5758	0.5750	0.5530
Lasso Regression	0.5331	0.5545	0.7446	0.5769	0.5760	0.5531
Random Forest	0.3275	0.2554	0.5053	0.8051	0.8047	0.6561

Table 1: Test-set performance and 5-fold cross-validated R^2 of all models (best row highlighted)

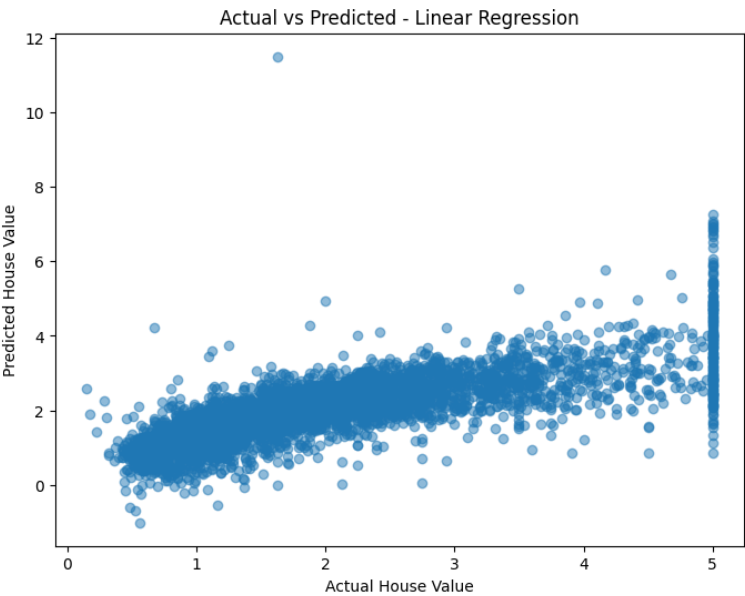


Figure 2: Actual vs predicted house value (Linear Regression)

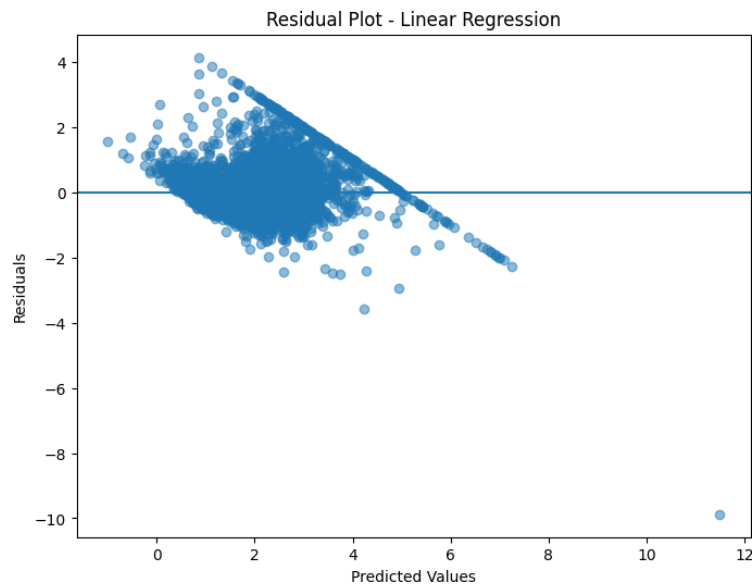


Figure 3: Residual plot (Linear Regression)

4.1 Analysis

- **Random Forest is clearly best:** R^2 of 0.805 against about 0.576 for the linear family, and an RMSE of 0.505 against about 0.745 (roughly \$50,500 vs \$74,500 typical error). Its MAE drops by about 39% (0.533 $\rightarrow$ 0.328).
- **Linear, Ridge and Lasso are almost identical.** With 16,512 samples and only 8 features the linear model is not overfitting, so regularisation has almost nothing to correct. The remaining error is caused by **bias** (underfitting), not variance. The data contains non-linear effects (for example location) that a straight-line model cannot represent.
- **Residual diagnostics:** the residual plot is not randomly scattered around zero. The spread changes with the predicted value (heteroscedasticity), and a diagonal band appears at high values because the target is capped at 5.0. The actual-vs-predicted plot shows the same flattening of the predictions at the top end. This confirms that the linearity and constant-variance assumptions are violated.
- **Adjusted R^2** is very close to R^2 for every model, so the 8 features are not inflating the fit.
- **Cross-validation gap:** the Random Forest CV R^2 (0.656) is noticeably lower than its test R^2 (0.805), whereas the linear models agree closely (0.553 vs 0.576). The folds were created without shuffling, and the dataset is ordered by geographic block. Each fold may therefore represent a different region, which is harder for a location-sensitive model to generalise to. The test-set score can be optimistic, and the true generalisation performance is probably somewhere between the two figures. Shuffled K-fold would give a fairer estimate.
- **Trade-off:** Random Forest is more accurate but less interpretable and more expensive to train and to predict with. The linear models give interpretable coefficients and are very fast.

5. Task 2: Gradient Descent for Linear Regression

5.1 Theory

Linear Regression predicts $\hat{y} = Xw + b$. Gradient Descent minimises the cost function iteratively:

$$\begin{aligned} J(w, b) &= (1 / 2m) \cdot \sum (\hat{y}_i - y_i)^2 \\ \partial J / \partial w &= (1/m) \cdot X^T(\hat{y} - y) & \partial J / \partial b &= (1/m) \cdot \sum (\hat{y} - y) \\ w &:= w - \alpha \cdot \partial J / \partial w & b &:= b - \alpha \cdot \partial J / \partial b \end{aligned}$$

Here α is the learning rate and m the number of training samples. Because the cost function of Linear Regression is convex, Gradient Descent converges to the single global minimum, which is the same solution as the Normal Equation $w = (X^T X)^{-1} X^T y$.

5.2 Implementation

A class `GradientDescentLinearRegression` was written from scratch using NumPy (batch gradient descent). Weights and bias are initialised to zero, the cost is recorded at every epoch, and the parameters are updated with the gradients above. The model was trained on the standardised training data with $\alpha = 0.01$ for 1000 epochs. It was then compared with the Normal Equation, computed with the pseudo-inverse: $\theta = \text{pinv}(X^T X) \cdot X^T y$.

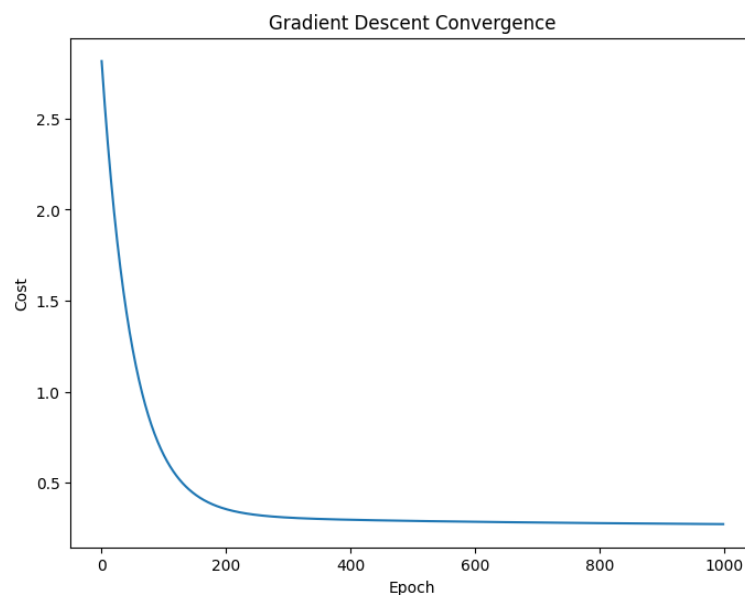


Figure 4: Gradient Descent convergence ($\alpha = 0.01$, 1000 epochs)

5.3 Effect of the Learning Rate

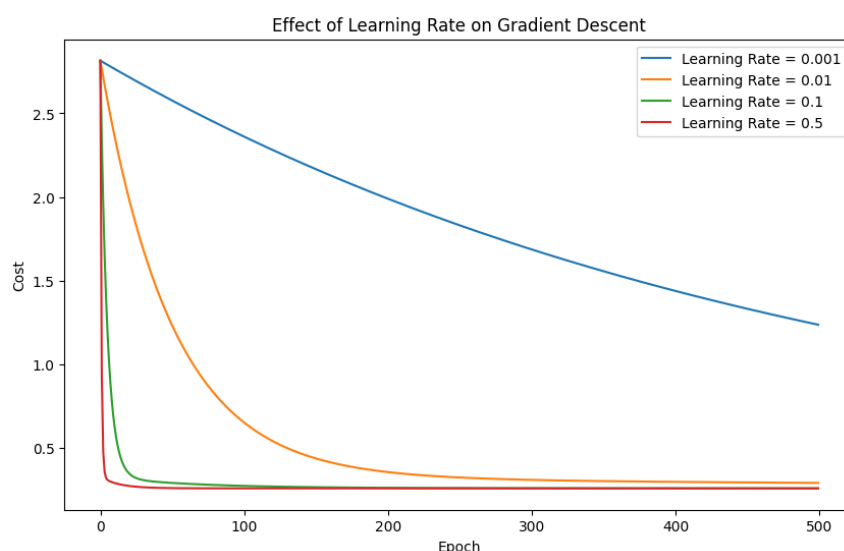


Figure 5: Effect of learning rate on cost (500 epochs)

- $\alpha = 0.001$: very slow. After 500 epochs the cost is still about 1.24, far from the minimum, so it is still underfitting.
- $\alpha = 0.01$: converges smoothly but still slowly. The cost is about 0.29 after 500 epochs and is nearly flat.
- $\alpha = 0.1$ and 0.5 : converge within about 30 epochs to the minimum (cost about 0.26) with no divergence or oscillation. On standardised data, these larger rates are the most efficient choices.

A rate that is too small wastes computation, and one that is too large causes oscillation or divergence. The stable range here is wide because the features are standardised, which shows why feature scaling matters for Gradient Descent.

5.4 Comparison of Methods

Method	MAE	MSE	RMSE	R^2	Adj. R^2
Gradient Descent ($\alpha=0.01$)	0.5477	0.5672	0.7531	0.5672	0.5663
Normal Equation	0.5332	0.5559	0.7456	0.5758	0.5750
Linear Regression (sklearn)	0.5332	0.5559	0.7456	0.5758	0.5750

Table 2: Gradient Descent against closed-form solutions

The Normal Equation reproduces scikit-learn exactly ($R^2 = 0.5758$), confirming the implementation is correct. The Gradient Descent model reaches $R^2 = 0.5672$, about 0.009 lower. This is not a flaw in the method: with $\alpha = 0.01$ and 1000 epochs the algorithm had not fully converged (see Figure 5, where the curve is still slightly above the minimum). Using $\alpha = 0.1$ (as Figure 5 suggests) should give a result essentially equal to the closed-form solution.

Aspect	Gradient Descent	Normal Equation
Learning rate	Must be tuned	Not needed
Iterations	Many	One step

Aspect	Gradient Descent	Normal Equation
Complexity	$O(k \cdot m \cdot n)$, scales to large data	$O(n^3)$ matrix inversion; slow for many features
Feature scaling	Required for fast convergence	Not required
Applicability	Works for any differentiable model (logistic regression, neural networks)	Only linear least squares

6. Consolidated Results and Critical Evaluation

Model	MAE	RMSE	R^2	Adj. R^2	Remark
Linear Regression	0.5332	0.7456	0.5758	0.5750	Baseline
Ridge Regression	0.5332	0.7456	0.5758	0.5750	No gain
Lasso Regression	0.5331	0.7446	0.5769	0.5760	Marginal gain
Random Forest	0.3275	0.5053	0.8051	0.8047	Best
Gradient Descent	0.5477	0.7531	0.5672	0.5663	Not fully converged
Normal Equation	0.5332	0.7456	0.5758	0.5750	Exact solution

Table 3: Final comparison of all models

- **Suitability:** Random Forest is the most suitable model for this dataset because the price relationship is non-linear and spatially dependent. Linear models are suitable only when interpretability matters more than accuracy.
- **Limitations:** the target is capped at 5.0; the data come from 1990; a single random train/test split was used; hyperparameters (alpha, n_estimators) were not tuned; and the outliers in AveRooms/AveOccup were not treated.

6.1 Proposed Improvements

- Hyperparameter tuning with GridSearchCV/RandomizedSearchCV (Ridge/Lasso alpha, Random Forest depth and number of trees).
- Feature engineering: rooms per person, bedroom ratio, distance to major cities (from Latitude/Longitude), and polynomial features for MedInc.
- Try Gradient Boosting (XGBoost / LightGBM) and compare with Random Forest.
- Use shuffled K-fold cross-validation, and treat or remove the outliers and capped targets.
- Run Gradient Descent with $\alpha = 0.1$ for more epochs, add mini-batch / stochastic variants, and use learning-rate decay.

7. Ethical and Societal Considerations

- **Fairness:** price models trained on historical data can reproduce historical inequality (for example redlining) and undervalue homes in disadvantaged neighbourhoods. If used for lending or insurance this could lead to discriminatory decisions.

- **Proxy variables:** Latitude and Longitude can act as proxies for sensitive attributes such as race or income. Error should be audited across regions.
- **Transparency:** black-box models such as Random Forest should be accompanied by explanations (feature importance, SHAP) when they affect people's finances.
- **Data relevance:** the dataset is from 1990 and aggregated by block group, so it should not be used to price individual properties today without retraining on current data.

8. Conclusion

Four regression models were developed and evaluated on the California Housing data. Random Forest outperformed the linear models substantially (R^2 0.805 vs about 0.576, RMSE 0.505 vs about 0.745), showing that the problem is non-linear. Regularisation gave negligible benefit because the linear models suffer from bias, not variance. The from-scratch Gradient Descent implementation behaved as theory predicts: the learning rate strongly affects convergence speed, and $\alpha = 0.1$ – 0.5 converged fastest on standardised features. With $\alpha = 0.01$ it reached $R^2 = 0.567$, slightly below the closed-form R^2 of 0.576, because it had not yet fully converged. The Normal Equation matched scikit-learn exactly, validating the implementation.

Self-reflection

This assignment showed me that accuracy metrics alone are not enough: residual plots and cross-validation revealed problems that a single R^2 hides. Implementing Gradient Descent by hand made the role of the learning rate and of feature scaling concrete. In future work I want to explore hyperparameter tuning, boosting methods and model explainability.

References

- Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3), 291–297. (Origin of the California Housing dataset)
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- scikit-learn documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html